Working On It and Defending It

Part 5 of 5: running it, extending it, and answering for it

SETUP · ADDING A FEATURE · TROUBLESHOOTING · DEFENCE QUESTIONS

---

1. Running the project

Prerequisites

- .NET 10 SDK (this machine has it at `%LOCALAPPDATA%\Microsoft\dotnet`)
- Docker, for PostgreSQL

Start the database

```
docker compose up -d
```

`compose.yaml` runs `postgres:16-alpine`, exposing container port 5432 on **host port 5434** — which is why the connection string says 5434. The data lives in a named volume, so it survives `docker compose down`.

Configure secrets

`appsettings.json` contains `CHANGE_ME` placeholders and is committed to git, so real values must go somewhere else. Use user secrets, which are stored outside the repository:

```
dotnet user-secrets set "ConnectionStrings:Default"
"Host=localhost;Port=5434;Database=EduCoreDB;Username=educore;Password=educore123"
dotnet user-secrets set "Jwt:Key" "a-long-random-string-of-at-least-32-bytes"
dotnet user-secrets set "SuperAdmin:Email" "you@example.com"
dotnet user-secrets set "SuperAdmin:Password" "SomethingStrong123"
```

Configuration is layered: `appsettings.json`, then `appsettings.Development.json`, then user secrets, then environment variables — each overriding the last. Nothing sensitive is ever committed.

Create the schema and run

```
dotnet ef database update
dotnet run
```

On first boot `SuperAdminSeeder` creates the single SuperAdmin from configuration, if none exists. It is idempotent, so it is safe on every start. That account is your way in — log in as it, create a school, then create an admin for that school.

A first end-to-end test

1. `POST /api/auth/login` as the SuperAdmin → copy the token.
2. `POST /api/schools` with `Authorization: Bearer <token>`.

3. `POST /api/admins` for that school.
4. Log in as the new admin → the token now carries a `school_id` claim.
5. `POST /api/academic-years`, then `/api/subjects`, `/api/classes`, `/api/teachers`, `/api/students`.

That order is forced by the data model: a class needs an academic year, an enrollment needs a class and a student, a lesson needs a teaching assignment.

2. Adding a feature, end to end

Suppose you add *homework assignments*. The layers tell you the order, and it is the same every time:

1. **`Models/Homework.cs`** — plain properties. Implement `ISchoolScoped` if it belongs to a school (it does).
2. **`Data/AppDbContext.cs`** — add `DbSet<Homework>`, configure relationships in the right `Configure...` method, and call `ScopeToSchool<Homework>(modelBuilder)`. *Forgetting that one line means no tenant isolation on the new table.*
3. **`DTOs/Homework/`** — `HomeworkCreateDto`, `HomeworkListDto`. Records.
4. **`Validation/Homework/`** — a validator. Registered automatically by assembly scanning.
5. **`Mapping/MappingProfile.cs`** — `CreateMap<Homework, HomeworkListDto>()` and `CreateWriteMap<HomeworkCreateDto, Homework>()`.
6. **`Repositories/`** — `IHomeworkRepository` + `HomeworkRepository`, only for queries the generic base cannot do.
7. **`Services/`** — `IHomeworkService` + `HomeworkService`. Business rules and the `AccessGuard` calls live here.
8. **`Endpoints/`** — routes and policies.
9. **`Program.cs`** — register the repository and service, and call `api.MapHomeworkEndpoints()`.
10. **Migration** — `dotnet ef migrations add AddHomework`, then `dotnet ef database update`.

> **The checklist that prevents the two easy mistakes:** did you call `ScopeToSchool<T>`? And does every service method that takes an id from the client call an `AccessGuard` method? Those are exactly the two things that were missed at least once while building this.

3. Troubleshooting

SYMPTOM	CAUSE
`Failed to connect to 127.0.0.1:5434`	Postgres is not running. `docker compose up -d`.
`Jwt:Key must be at least 32 bytes`	Secrets not configured. See section 1.
Every list comes back empty	Your token has no `school_id`. You are logged in as SuperAdmin, whose school-scoped reads correctly return nothing.
`Cannot save X for school ... while acting for school ...`	`SchoolScopeInterceptor` caught a cross-school write. You set `SchoolId` from the request body instead of the token.
An edit “succeeds” but nothing changes	You modified an `AsNoTracking` entity. Load it tracked.
`The LINQ expression could not be translated`	Usually sorting on a property of an already-projected record. Sort before the projection.
`Cannot write DateTime with Kind=Unspecified`	Npgsql requires UTC for `timestamptz`. Use `DateTime.SpecifyKind(..., Utc)`.
409 on a create that looks fine	A unique index. Names are unique per school; personal numbers are unique globally.

4. Defence questions

Why multi-tenant with one database instead of one database per school?

Far simpler to operate: one schema, one migration run, one connection pool. The cost is that isolation becomes the application's responsibility, which is why it is enforced in two independent places — query filters for reads, an interceptor for writes — rather than trusted to developers remembering a `WHERE` clause.

How do you know School A cannot see School B's data?

Three things. Every school-scoped entity has a global query filter, so *every* generated query includes the school predicate — you cannot forget it because you never write it. Every write passes through `SchoolScopeInterceptor`. And the schema uses composite foreign keys through `(Id, SchoolId)` alternate keys, so a cross-school reference is rejected by PostgreSQL itself. The escape hatch is `.IgnoreQueryFilters()`; there are nine calls, all listed and justified in the security document.

Why do repositories exist when EF Core is already a repository and unit of work?

This is the strongest challenge to the architecture and it deserves an honest answer. `DbSet<T>` *is* a repository and `SaveChanges` *is* a unit of work, so this layer does wrap an existing abstraction. Two symptoms show it leaking: calling `SaveChangesAsync` on one repository persists entities loaded through another, because they share a `DbContext`; and `IAdminRepository.GetTrackedAsync` had to be invented to re-expose EF's tracking, the exact concept the layer was meant to hide. I keep it because it holds the query code in one place per entity, keeps services readable, and is the pattern most .NET codebases use. Dropping it would remove roughly 800 lines. It is a trade-off, not an obvious win either way.

Why interfaces on every service when each has one implementation?

It is how DI is conventionally wired in .NET, and it is the seam unit tests need — a test can supply a fake `IAccessGuard` without a database. There are no tests yet, so today it is mostly convention; retrofitting it later is more work than keeping it now.

What is an interceptor and why use one?

A hook EF Core calls at defined points in its own pipeline — here, just before and around `SaveChanges`. Both concerns they handle are cross-cutting: tenant checks and audit logging apply to *every* write. Putting them in an interceptor means no service can forget them. The alternative is the same two lines repeated in twenty methods, where one will eventually be missing.

Why is almost nothing deletable?

School records are historical and often legal documents. Every relationship is `Restrict`, and the model "ends" things instead: `DeactivatedAt`, `LeftAt`, `EndedOn`, `GraduationDate`. A student who transfers keeps their marks. A teacher who leaves is still named on the lessons they taught. Filtered unique indexes make this work — only one *active* enrollment per class, while ended ones accumulate.

Walk me through the composite keys.

Two plain foreign keys, `StudentId` and `ClassId`, are each individually valid but permit a School A student in a School B class. So `Student` and `Class` get alternate keys on `(Id, SchoolId)`, and `Enrollment`'s foreign keys point at those pairs using its own `SchoolId` for both. The same value has to match both sides, so the database rejects the mismatch. `Attendance` uses the identical trick with `ClassId` to force the enrollment's class and the lesson's class to be the same.

Why can a substitute log a lesson but not give a mark?

Because they are different acts. Recording that a lesson happened is a factual observation, and the person who covered the class is the right person to make it — so a lesson only requires subject *qualification* (`TeacherSubject`). Grading is an authority judgement, so a mark requires the active *assignment* (`ClassSubjectTeacher`), enforced both in `MarkService` and by a five-column foreign key.

How are ids generated?

EF generates a `Guid` in C# when you call `AddAsync`, not the database. That timing lets one `SaveChanges` insert a lesson and its whole attendance register together, and lets audit rows join the same transaction. EF's generator produces increasing values so index inserts append rather than scatter.

What happens if two people edit the same record?

Every entity has a shadow `Version` property mapped to PostgreSQL's `xmin`. EF includes the original value in the `UPDATE`'s `WHERE`; if someone else has written since, zero rows match and EF throws `DbUpdateConcurrencyException`. Honest caveat: that is not yet mapped to a 409, so it would currently surface as a 500.

Why AutoMapper rather than writing the mapping by hand?

It removes repetitive entity-to-DTO copying, and `CreateWriteMap` makes inbound mapping safe by ignoring every destination property the DTO does not name — so `Id` and `SchoolId` stay under the service's control. The trade-off is real: mapping mistakes surface at runtime rather than compile time, which is why `AssertConfigurationIsValid()` runs at startup in development.

What is the weakest part of the project?

That it has never run against a live database. It compiles cleanly, DI validates and AutoMapper's configuration is verified at boot, but no endpoint has been exercised end to end. After that: no automated tests, no token revocation on deactivation, and no rate limiting on login.

What would you do next?

In order: run it against Postgres and walk the four flows by hand; add integration tests for tenant isolation specifically (log in as School A, try to read School B, assert 404/403); add the `SecurityStamp` so deactivation takes effect immediately; add rate limiting on login; map `DbUpdateConcurrencyException` to 409.

5. Which parts are genuinely advanced

Being straight about this is a strength, not an admission of weakness. If you are asked how much of it you would have written unaided after a week, the honest answer is: the endpoints, DTOs, validators and most services, yes. The three items below are the ones to be candid about — understand what they do and why, and say plainly that you would need to look them up to write again from scratch.

FEATURE	WHY IT IS ADVANCED	SIMPLER ALTERNATIVE IF PUSHED
Composite-key tenancy	Alternate keys and multi-column FKs are not beginner EF Core.	Plain FKs plus service-layer checks. Less safe, much more ordinary.
Save-changes interceptors	Requires knowing EF's internal pipeline.	Call an audit helper and a tenant check explicitly in each service.
Global query filters	Invisible behaviour — SQL you never wrote.	An explicit `.Where(x => x.SchoolId == currentUser.SchoolId)` in every query.

The simpler alternatives are all genuinely worse — they move a guarantee from "impossible" to "nobody has forgotten yet". But knowing the trade-off *is* the understanding. Being able to say "I chose the stronger option, here is what the weaker one would cost" is a better defence than having written the weaker one yourself.

6. A study plan

1. **Read `Program.cs` top to bottom.** It is the table of contents: every service, every policy, every filter, in order.
2. **Trace one request in the debugger.** Breakpoint in an endpoint, step into the service, into the repository. Watch the SQL in the console.
3. **Break something on purpose.** Comment out a `ScopeToSchool` line and watch another school's rows appear. Nothing teaches what a query filter does faster.
4. **Explain the four walkthroughs out loud** without the document.
5. **Write one new endpoint yourself** using the checklist in section 2. Something small — a list of subjects with teacher counts.

Step 3 and step 5 are the ones that convert reading into ownership. Do those two even if you skip the rest.